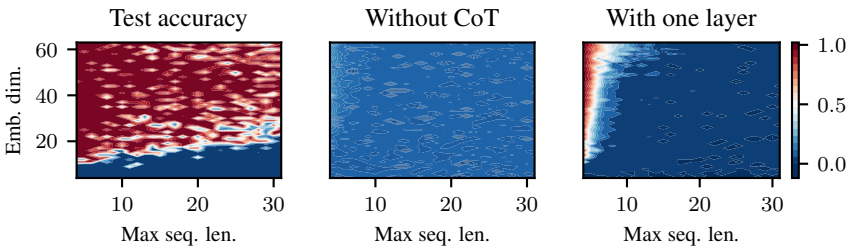

Figure 7: Test accuracy (where red indicates better performance) after learning the polynomial iteration task with $P(X, Y) = XY + 1$ in $\mathbb{F}_{11}$ for 1000 epochs. The accuracy is reported as a function of the embedding dimension (on the y-axis), and the maximum sequence length $L_{\max}$ (on the x-axis). The learning was conducted with a two-layer transformer with CoT (left), without CoT (middle), or with a one-layer transformer with CoT (right). This illustrates the usefulness of CoT and two-layer architectures.

Figure 6. Using the information of x_t and s_t, the following MLP can then compute $s_t = F(x_t, s_t)$. For a given sequence length L, the learned attention maps were found to be invariant to the input token $(x_t)_{t\in[L]}$: the standard deviation of attention patterns computed over all the data was negligible.

Successor Function. We empirically verified that after training a two-layer transformer with one attention head per layer on the copying dataset, fine-tuning only the second layer MLP on parity data enabled us to achieve 100% accuracy on the parity problem. In the context described previously, this transfer was accomplished in fewer than 20 epochs of fine-tuning on the parity dataset. This confirms that the feed-forward layer of the second transformer block is computing the successor function F. In more general contexts, we found the successor function to be implemented jointly by the second layer MLP, the second attention values and output matrices, as well as the un-embedding matrix.

Position Subtraction. The accuracy of the model decreases with the embedding dimension d, as shown on the left of Figure 7. Figure 8 suggests that when d is small, the first attention layer remains capable to accurately locate the "EoI" token, but the second attention layer struggles to retrieve x_t. This can be explained as follows: in a model that implements an iteration head, the first layer's MLP, in conjunction with the second attention key and query matrices, is expected to generate the query-key pair "Are you p_t" and "I am p''_t by transforming p_t on the one hand, and a superposition of p_{L+t} and p_{L+1} on the other hand, so that the end results are aligned. More abstractly, this encodes the positional subtraction $L + t - (L + 1) + 1 = t$. In high dimensions, it is relatively easy to find a set of weights to align a large number of vectors (viz., the ones encoding for (p_{L+1}, p_{L+t}) and for p_t). However, in lower dimensions, this can only be achieved when the vectors form certain special geometrical patterns [see e.g. 40, 63], which the model struggles to learn in our setting, at least with the optimization choices we made.

Evaluation Dynamics. With a sufficiently small model, we might expect to understand the training dynamics quite well, which could in turn provide insights on design choices for larger models to minimize training costs. While a detailed study of the training dynamics of our two-layer transformers is beyond the scope of this paper [see e.g., 41], we note several interesting facts that align with recent findings in the literature, such as the staircase profile of accuracy plots in Figure 6 [1, 3, 13], as well as the usefulness of small batch sizes and large learning rates reported in Figure 12 in Appendix [11] despite the risk of loss spikes [12, 59].

3.3 Ablation Studies

In addition to visualizing the attention map, we validated the learning of iteration heads through attention patching, i.e., intervening to "patch" certain attention maps. Specifically, we observed that patching the ideal attention maps (i.e., zeroing out other routes) does not disrupt perfect accuracy. In contrast, zeroing out the focus on the EoI by the first attention head, or on p_t by the second, reduced performance to near random.

Next-token Prediction; One or Two Layers. As an initial ablation study, we considered the polynomial iteration problem with $P(X, Y) = XY + 1$ in $\mathbb{F}_{11}$, and compared the performance of

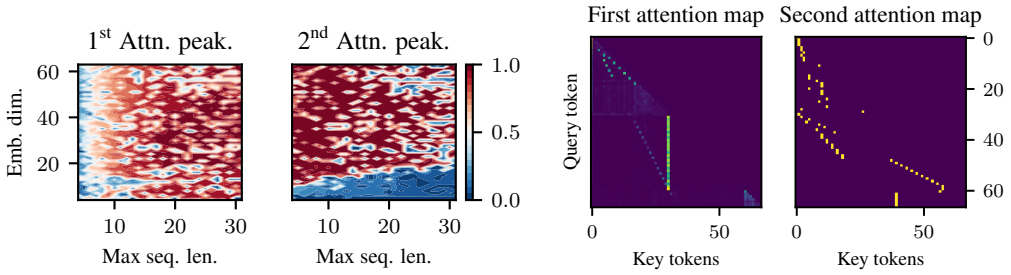

Figure 8: Left: attention peakiness score after 1000 epochs of learning with the polynomial iteration task parameterized by $P(X, Y) = XY + 1$ in $\mathbb{F}_{11}$ as a function of the embedding dimension d and the maximum sequence length $L_{\max}$. Right: example of attention maps of sub-sampled iteration heads.

CoT reasoning with next-token prediction (i.e., without CoT), as well as CoT with a single layer transformer. Two parameters come into play: the length of the sequence, which can be seen as a difficulty parameter regarding the data; and the embedding dimension, which can be seen as a model capacity parameter [31, 55, 53]. The results, unequivocal in favor of CoT and two-layer transformers, are reported in Figure 7.

Alternative Circuits. Next, we explored other circuits that a two-layer transformer can learn to perform the same tasks as an iteration head. We proceed by assigning a score to measure how closely the attention maps follow the patterns of Figure 6. For the first attention map, we would like a measure of the concentration of the attention at the "Are you EoI? I am" query-key pairs, which correspond to the vertical yellow line from the left of Figure 6. For the second attention map, we would like a measure of the concentration of the attention at the "Are you p_t? I am" query-key pairs, which corresponds to the yellow off-diagonal from Figure 6. To avoid scaling issues, we define an attention score a_i as "peaky" if it is greater than 50% after softmax averaging. We then measure the average number of peaky scores (within one sequence, and over sequences), i.e., we compute $\sum \mathbf{1}_{a_i > .5}$ instead of $\sum a_i$. This provides a clear measure of the degree to which a transformer is implementing the attention mechanisms described in the previous section.

On the left of Figure 8, we report the average peakiness found for the first and second attention layers when training a transformer for different maximum sequence lengths $L_{\max}$ and embedding dimensions d. We only run one experiment per pair $(L_{\max}, d)$ with a fixed random seed. The texture of the figure indicates a certain randomness between runs for similar pairs $(L_{\max}, d)$. The first attention layer almost always learns the "Are you EoI?" query-key combination, except when the maximum sequence lengths are very small. In these cases, the transformer might find different circuits to solve for different sequence lengths.

The second attention layer tends to vary more. In particular, for small embedding dimensions, the position subtraction might be challenging for the transformer to perform, leading it to find alternative mechanisms. For instance, the first layer attention might perform a previous token copy when processing the input tokens, superposing the current token x_t and the previous one x_{t-1} in the current working space. This allows the second layer to solely point at every other position, e.g., only attend even positions $t \in 2 \cdot \mathbb{N}$, either recovering the current token x_t, or the previous one x_{t-1}. Implementing position subtraction towards even positions only reduces the learning capacity needed by the first layer MLP [see 11, for related scaling laws]. Such a sub-sampling mechanism is notably observed on the right of Figure 8.

4 Skill Transfer

Some skills might be easier to acquire when trained on certain data rather than others, highlighting the importance of data curation when training LLMs. For example, datasets of code or math [e.g., 26] might exhibit formal reasoning structures that compel LLMs to learn multistep reasoning patterns when trained on them, leading to improved reasoning abilities of the final model, even in plain English [see, e.g., 37]. Our synthetic problems are ideally suited to highlight the mechanisms at play in these observations. This section illustrates how strategic data curation can facilitate learning to solve the parity problem. The crux is to find a dataset that helps the creation of iteration heads, which, once present, significantly eases the learning of the parity problem by a transformer.

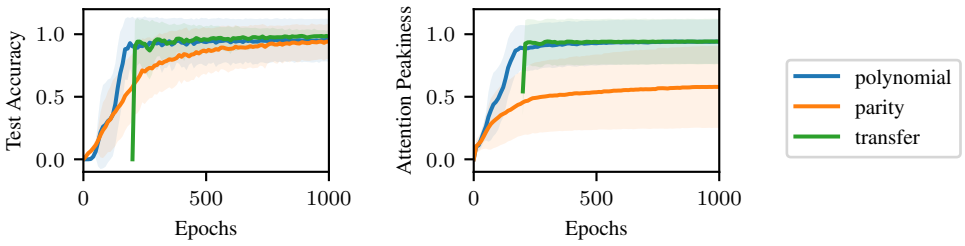

Figure 9: Left: Test accuracy as a function of the number of epochs, averaged over 100 runs, when learning the polynomial iteration task with $P(X, Y) = XY + 1$ in $\mathbb{F}_{11}$ (blue) and the parity problem (orange). Right: The second attention peakiness score indicates whether the network is learning the iteration head described in Figure 5. The green curve corresponds to the accuracy on the parity problem when learning the polynomial iteration for the first 200 epochs before switching the dataset to learn the parity problem.

4.1 Inducing Induction

We now address a simple question: can we "pretrain" a model on a task A, and then "finetune" it on a task B in order to learn to solve the task B with a smaller total number of flops than if we were to learn the task B from scratch? We will see that the answer is positive.

Figure 9 compares three learning scenarios. The learning of the polynomial iteration task corresponding to $P(X, Y) = XY + 1$ in $\mathbb{F}_{11}$ is reported in blue. The learning of the parity problem is reported in orange. Finally, the green curve represents training on the polynomial iteration task for 200 epochs (these epochs are not reported in the graph, hence the curve offset), before switching tasks and continuing the training on the parity problem. When switching from the polynomial iteration dataset to the parity dataset, we chose to reset the Adam buffers to zero. Moreover, our default experimental parameters were changed to $L_{\max} = 16$ and $n = 512$, generating training and testing sets of $N = 8,192 = 16 \times 512$ sequences. The left side of Figure 9 reports testing accuracy averaged over 100 runs, along with its standard deviation. The polynomial iteration task is learned relatively quickly, while the parity problem takes longer. The right side of Figure 9 reports the second attention peakiness score, capturing whether or not the second attention is implementing the "Are you p_t?" query. After 200 epochs of training with the polynomial iteration task, the iteration head is formed, and fine-tuning the network on the parity problem for less than 30 epochs enables the reuse of this circuit on the parity data (green curve, right plot), thus solving the parity task (green curve, left plot). Overall, the data curation represented by the green plot enables the computation of parities in less than 300 epochs, compared to 1000 epochs when learning solely with parity data.

This example provides a controlled setup to understand the usefulness of data curation when training larger models. It biases the model toward the implementation of specific circuits. In particular, adding code or math datasets to the training of LLMs might induce the learning of more circuits that implement various forms of reasoning patterns. These could be viewed as atomic skills that could be reused to solve more generic problems [see, e.g., 4, for further discussions on skill factorization].

4.2 The Role of Inductive Biases

To illustrate the usefulness of data curation and skill transfer, we needed to find a problem that is hard to learn from scratch. The parity problem was well-suited to play this role in our synthetic setting. On the other hand, the polynomial iteration task with $P(X, Y) = XY + 1$ in $\mathbb{F}_{11}$ was the easier task. One might wonder why learning with $P(X, Y) = XY + 1$ in $\mathbb{F}_{11}$ turned out to be a simpler task than learning parities, which corresponds to $P(X, Y) = X + Y$ in $\mathbb{F}_2$. Our intuition is that the parity problem can be solved in many different ways, which leads to competing signals in the gradient for updating the weights, reminiscent of the theoretical study by Shalev-Shwartz et al. [52] [see also 50, 63]. For example, we see on the right of Figure 9 that the standard deviation of the attention peakiness score is quite high when learning with parity data. This can also be observed from the texture in Figure 11 in the Appendix. This creates a *challenging optimization landscape*. In contrast, the polynomial $P(X, Y) = XY + 1$ was chosen to make the final state dependent on the token order. Removing permutation invariance is useful to reduce the variety of circuits that can solve the polynomial iteration task, and seems to speed up the training dynamics. Finally, starting from a pretrained model that already implements an iteration head creates a strong inductive bias toward the